

**EMOTION-AWARE AGILE SYSTEM FOR PROGRESS
TRACKING, EXPERTISE RECOMMENDATION,
REQUIREMENT MANAGEMENT AND INCLUSIVE
COMMUNICATION**

Project ID: 25-26J-525

Project Proposal Report

Pethmi Induma Vithana

IT222136688

Supervised by – Ms. Ishara Weerathunga

B.Sc. (Hons) in Information Technology Specializing in
Software Engineering

Department of Computer Science & Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

**EMOTION-AWARE AGILE SYSTEM FOR PROGRESS
TRACKING, EXPERTISE RECOMMENDATION,
REQUIREMENT MANAGEMENT AND INCLUSIVE
COMMUNICATION**

Project ID: 25-26J-525

Project Proposal Report

B.Sc. (Hons) in Information Technology Specializing in
Software Engineering

Department of Computer Science & Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

DECLARATION

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Vithana P.I.	IT22136688	

The supervisor/s should certify the proposal report with the following declaration.
The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

Signature of the Supervisor: (Ms. Ishara Weerathunga) 	Date 30/8/2025.
---	-----------------

Abstract

Agile software development has become the dominant methodology for modern teams, but existing tools often overlook the emotional dynamics that directly influence productivity, collaboration, and sprint outcomes. This study aims to address that gap by developing an Emotion-Aware Agile System that integrates emotional intelligence into progress tracking, expertise recommendation, requirement management, and inclusive communication. The research problem focuses on how unrecognized emotional states—such as frustration, demotivation, or burnout—lead to inefficiencies in Agile workflows and conflict within teams.

The proposed solution adopts a multi-module AI architecture combining Natural Language Processing (NLP), facial and micro-expression recognition, semantic embeddings, and predictive analytics. The system includes an emotion detection engine to classify team members' emotions from chat and video data, an Agile optimization module that maps emotional signals to metrics such as velocity and workload distribution, an expertise recommendation system that uses developer embeddings and task similarity scoring for intelligent task assignment, a sprint replanning tracker that predicts the impact of requirement changes and generates revised backlogs, and an inclusive brainstorming assistant that rephrases and enhances hesitant inputs to ensure equal participation. Together, these modules form a proactive decision-support assistant for Scrum Masters and Product Owners.

Through this design, the project aspires to build the first integrated framework that links emotional intelligence with Agile practices in hybrid English-speaking development teams. The anticipated contribution is a novel system that not only monitors but also responds to emotional cues in real time, offering actionable recommendations for workload balancing, conflict mitigation, and inclusive communication. This research advances the intersection of affective computing and Agile software engineering, with the potential to create more resilient, collaborative, and human-centered development environments.

Keywords: *emotion detection engine, Agile optimization, expertise recommendation, sprint replanning tracker, inclusive brainstorming assistant*

Table of Contents

DECLARATION.....	1
Abstract	2
1. INTRODUCTION	5
1.1. Background and Literature Survey	6
1.2. Research Gap	7
1.3. Research Problem.....	9
2. OBJECTIVES	10
2.1. Main Objective.....	10
2.2. Specific Objective	11
3. METHODOLOGY	12
3.1 System Architecture	12
3.2. Component Architecture	14
3.3 Technologies.....	14
4. PROJECT REQUIREMENTS	16
4.1 Functional Requirements	16
4.2 Non-Functional Requirements	17
4.3 System Requirements.....	17
4.4 User Requirements	18
5. DESCRIPTION OF PERSONAL AND FACILITIES.....	19
6. BUDGET AND BUDGET JUSTIFICATION	20
7. REFERENCES.....	21

List of Figures

Figure 3.1: System Diagram.....	9
Figure 3.2: Component System Diagram.....	10
Figure 5.1: Work Breakdown Structure.....	15
Figure 5.2: Gantt Chart.....	15

List of Tables

Table 1.1: Existing Research Comparison.....	4
Table 6.1: Overall Budget For System.....	16

1. INTRODUCTION

Agile software development has emerged as the predominant methodology for contemporary software engineering teams, prized for its iterative nature, adaptability to change, and emphasis on collaborative dynamics [10]. While frameworks such as Scrum and Kanban provide robust structures for task management and workflow optimization, they predominantly address technical and procedural aspects, largely neglecting the critical human factors that fundamentally influence team performance and project outcomes. Research in organizational psychology and software engineering consistently demonstrates that emotional states—including frustration, demotivation, and burnout—significantly impact productivity, collaboration quality, and ultimately, sprint success [9]. Traditional Agile management tools (e.g., Jira, Trello) excel at tracking tangible metrics but remain fundamentally "emotion-blind," creating a critical gap in holistic project management.

The Requirement Change Impact & Sprint Replanning Tracker is a predictive, data-driven module designed to address a critical gap in modern Agile management. While existing tools like Jira excel at recording changes, they fail to anticipate their consequences. This component automatically detects requirement modifications mid-sprint, leverages machine learning to forecast their multi-faceted impact on effort, schedule, quality, and team morale, and provides proactive, actionable recommendations for dynamic replanning. By transforming reactive change tracking into proactive decision intelligence, it empowers teams to mitigate risks, optimize workflows, and safeguard sprint objectives with unprecedented foresight.

1.1. Background and Literature Survey

Agile teams routinely face requirement changes that arrive mid-sprint, creating uncertainty around delivery commitments, quality outcomes, and team workload. While mainstream lifecycle tools (e.g., Jira, Trello, Azure DevOps) are effective at recording backlog items and changes, they provide limited support for anticipating the impact of those changes on near-term sprint outcomes and do not automate dynamic replanning. Empirical and review literature consistently links unplanned change to increased schedule slippage, budget growth, and quality degradation—especially when scope evolves without structured impact analysis and mitigation [6], [7], [8].

Change effects are rarely uniform. They propagate along requirement dependencies, alter team effort/time distributions, manifest as schedule risk (e.g., spillovers), influence quality (defect injection/reopened issues), and can depress productivity/morale through backlog churn and reassignments. Techniques that explicitly model dependency structures have been shown to strengthen impact analysis and change prediction [5]. Likewise, evidence from Earned Value Management (EVM) demonstrates how plan deviations can be quantified early, but EVM by itself is retrospective unless coupled with predictive signals and proactive responses [6].

Recent research in data-driven Agile analytics suggests sprint-level representation learning can capture temporal patterns of work, velocity, and outcomes (e.g., Sprint2Vec) [1]. Reviews of Agile requirement change management (RCM) highlight process practices (backlog grooming, impact analysis, traceability), yet note gaps in automation, predictive guidance, and decision support—particularly in web engineering contexts and distributed settings [2], [3], [8]. In parallel, non-functional requirement (NFR) traceability models emphasize linking quality concerns to backlog items so that change-driven quality risks are visible and analyzable [4].

Your proposed Requirement Change Impact & Sprint Replanning Tracker is designed to close these gaps by: (i) continuously detecting change events in sprint data, (ii) engineering features around effort/time, schedule risk, quality, and productivity, (iii) predicting short-term impacts on velocity, delivery risk, defect likelihood, and workload, (iv) prioritizing high-impact items via a configurable scoring engine, and (v) proactively recommending mitigation and dynamically replanning the sprint. In doing so, it operationalizes insights from dependency-aware impact analysis [5], cost/schedule evidence from scope change literature [6], [7], and traceability-centric quality management [4], while leveraging modern sprint-level representations to improve predictive accuracy and timeliness [1], [2], [8].

1.2. Research Gap

Existing research has extensively addressed requirement change management (RCM) in Agile development, with emphasis on success factors, traceability, scope control, and impact measurement. However, several critical gaps remain:

1. **Limited predictive capability:** Most Agile tools (e.g., Jira, Trello) and RCM frameworks record changes and outcomes but lack predictive models for estimating how requirement changes will affect sprint velocity, defect likelihood, or delivery delays [2], [3], [8].
2. **Absence of proactive decision support:** Current studies highlight the importance of change detection and traceability [4], [5] but do not integrate recommendation systems that suggest mitigation strategies (e.g., splitting stories, adding buffers, rebalancing workload).
3. **Weak dynamic replanning support:** Scope change analysis with EVM [6] or cost impact assessment [7] remains retrospective, offering little real-time replanning guidance during sprints.
4. **Fragmentation of factors:** Research often addresses isolated aspects (dependencies [5], NFR traceability [4], or project cost [7]) but does not holistically integrate effort/time, schedule risk, quality, and productivity into a unified model.
5. **Lack of adaptive dashboards:** While systematic reviews [8] call for improved RCM tool support, existing works stop at reporting or visualization. They do not offer interactive, data-driven dashboards with drill-down insights and exportable reports for sprint replanning.

Your Requirement Change Impact & Sprint Replanning Tracker addresses these gaps by combining real-time change detection, feature engineering, impact prediction, priority scoring, proactive mitigation strategies, dynamic sprint replanning, and decision-support dashboards in one integrated system.

Existing Research vs. Proposed Component

Legend: ✓ = Feature supported; ✗ = Not supported / not primary focus

Features	A	B	C	Proposed Component
Change Detection (added/removed/modified items)	✗	✗	✗	✓
Impact Prediction (velocity, delay, defects, productivity)	✓	✗	✗	✓
Requirement Dependencies Handling	✗	✗	✗	✓
Quality/NFR Traceability & Risk Analysis	✗	✗	✗	✓
Effort/Time Risk Estimation (velocity, workload)	✓	✗	✗	✓
Schedule Risk Estimation (spillovers, delay probability)	✗	✗	✗	✓
Cost/Project Budget Impact	✗	✗	✗	✓
Proactive Risk Mitigation Recommendations	✗	✗	✗	✓
Dynamic Sprint Replanning (task redistribution, adaptive backlog)	✗	✗	✗	✓
Decision-Support Dashboard (visuals, reports, drill-down)	✗	✗	✓	✓
Exportable Reports & Stakeholder Communication	✗	✗	✓	✓

Table 1.1: Existing Research Comparison 1

A: Sprint2Vec [1]

B: Agile RCM in Web Eng. [2]

C: Success Factors in GSD [3]

1.3. Research Problem

Agile software development relies on short, iterative sprints where requirement changes are inevitable due to evolving stakeholder needs, shifting priorities, and emerging risks. While tools such as Jira, Trello, and DevOps platforms effectively record backlog items and sprint outcomes, they primarily provide static reporting and retrospective visibility. Existing academic studies have explored various aspects of requirement change management, such as sprint characterization [1], success factors in global software development [3], non-functional requirement (NFR) traceability [4], requirement dependency-based impact analysis [5], and scope/cost effects of requirement changes [6], [7]. However, these approaches remain fragmented, addressing isolated dimensions like cost, quality, or dependencies, and largely lack integrated predictive models or real-time adaptive planning mechanisms.

Specifically, there is no comprehensive solution that can:

1. Automatically detect requirement changes within sprint backlogs,
2. Predict their multi-dimensional impacts (effort/time, schedule risk, quality degradation, and team productivity), and
3. Proactively recommend mitigation strategies while enabling dynamic sprint replanning through interactive dashboards.

This gap leaves Agile teams vulnerable to unanticipated delays, reduced velocity, increased defect rates, and declining team morale, especially when requirement changes occur mid-sprint.

Therefore, the research problem addressed in this study is:

How can a predictive, data-driven system be designed to detect requirement changes during sprints, estimate their impacts across key Agile performance factors, and proactively support dynamic sprint replanning through actionable recommendations and decision-support dashboards?

2. OBJECTIVES

2.1. Main Objective

The primary objective of this research is to design and develop an integrated, emotion-aware AI assistant capable of enhancing both the productivity and well-being of Agile software development teams. Unlike traditional project management tools that focus solely on task tracking and performance metrics, this system seeks to bridge the gap between human emotional intelligence and process efficiency. It will achieve this by processing multimodal communication data collected from platforms such as Slack, GitHub, and Jira, in addition to real-time video and audio feeds during meetings and collaborative sessions. By analyzing these diverse sources, the AI assistant can detect subtle emotional cues, identify communication patterns, and understand team dynamics in context, offering a holistic view of team health that goes beyond conventional analytics.

Leveraging the insights derived from emotional and contextual data, the system will correlate these findings with core Agile performance indicators such as velocity, sprint completion rates, task dependencies, and quality metrics. This correlation will enable the assistant to provide proactive, data-driven recommendations for workload balancing, conflict resolution, and strategic sprint replanning. By delivering timely alerts, actionable guidance, and predictive analytics, the system aims to foster a more collaborative, efficient, and psychologically safe work environment. Ultimately, this research seeks to create a responsive, emotionally intelligent support tool that empowers Agile teams to manage human factors alongside technical objectives, ensuring sustainable performance, improved morale, and enhanced decision-making throughout the software development lifecycle.

2.2. Specific Objective

1. To develop an automated data acquisition module that interfaces with Agile project management tools via API to collect real-time data on sprint backlogs and historical outcomes.
2. To design a robust change detection mechanism that identifies additions, removals, and modifications to requirements, including changes in story points, priority, and dependencies.
3. To engineer a multifaceted machine learning model capable of predicting the impact of changes on effort & time, schedule risk, quality, and team productivity/morale.
4. To create a dynamic prioritization engine that synthesizes model predictions into a single, configurable priority score for each backlog item.
5. To generate proactive, actionable mitigation recommendations—such as story splitting, task resequencing, and workload redistribution—to address predicted risks.
6. To implement an intuitive decision-support dashboard that visualizes changes, predictions, and priorities to facilitate dynamic sprint replanning.

3. METHODOLOGY

The development of the Emotion-Aware Agile Assistant will follow a systematic, iterative methodology grounded in robust software engineering and AI principles. The first phase involves the acquisition and curation of multimodal datasets, including anonymized team communications from platforms like Jira and Slack, pre-existing facial image datasets, and Agile metrics pulled via API. This heterogeneous data will be rigorously preprocessed: text will be tokenized, images will be normalized and augmented, and time-series Agile data will be structured for temporal analysis to create a solid foundation for model training.

Subsequent phases focus on core AI development and systems integration. Individual sub-modules—including a CNN for facial emotion recognition, fine-tuned transformers for text-based emotion detection, and tree-based algorithms like XGBoost for predicting requirement change impacts—will be developed and validated independently. These components will then be unified into a scalable microservices architecture, containerized with Docker and orchestrated with Kubernetes. The final system will be accessed through an intuitive React.js dashboard, providing real-time analytics and decision-support, before undergoing comprehensive unit, integration, and user acceptance testing to ensure practical viability and robustness.

3.1 System Architecture

The system diagram illustrates an Agile Intelligence Platform composed of four independent yet complementary components: Emotion Monitoring, Expertise Recommendation, Sprint Replanning Tracker, and Inclusive Brainstorming. Each component operates autonomously but contributes to a holistic view of Agile team performance. The Emotion Monitoring System integrates multimodal data such as text (from messaging tools), facial micro-expressions (from video conferencing), and voice tone (from voice capture services) to detect emotions in real time. It tracks team mood evolution over multiple meetings, assigns confidence scores to emotional insights, and links these trends with Agile metrics like sprint velocity, backlog balance, and blocker resolution time. This predictive and context-aware fusion ensures reliable emotion insights, allowing Scrum Masters to make informed decisions regarding team dynamics and workload balance.

The Expertise Recommendation System processes data from Jira, GitHub, and DevOps tools to build semantic developer profiles using embeddings, enabling accurate task-to-expert matching with ranked recommendations. Meanwhile, the Sprint Replanning Tracker monitors backlog and requirement changes, predicting

their impact on velocity, schedule, quality, and productivity. It provides proactive mitigation strategies and generates revised sprint backlogs dynamically. Finally, the Inclusive Brainstorming Platform empowers team members to contribute ideas anonymously or with enhanced clarity, offering rephrasing support, diversity analytics, clustering, and summarization of inputs. Together, these independent modules ensure that Agile teams benefit from emotion-aware decision-making, expertise-driven task allocation, adaptive sprint replanning, and inclusive collaboration.

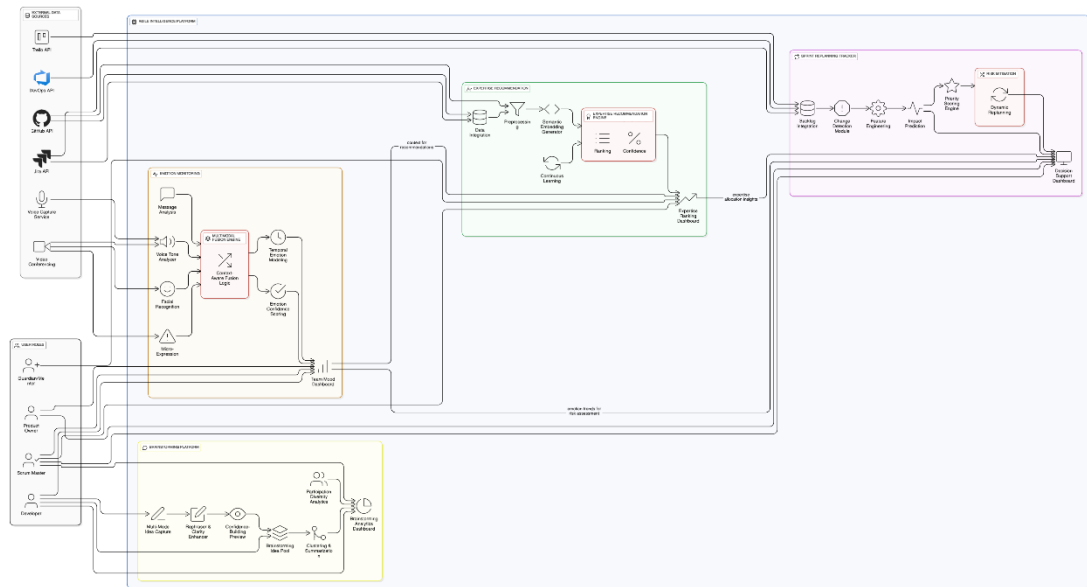


Figure 3.1: System diagram

3.2. Component Architecture

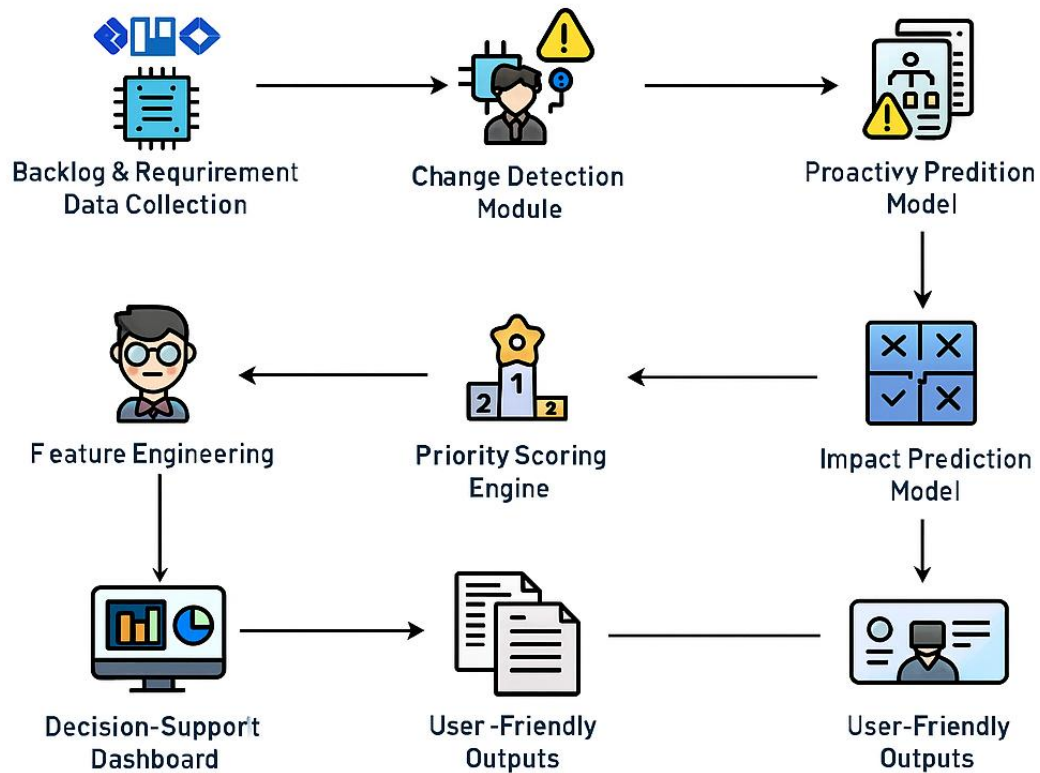


Figure 3.2: Component System Diagram

3.3 Technologies

The development of the Emotion-Aware Agile Assistant will leverage a modern, robust technology stack to handle the complex demands of multimodal data processing, machine learning, and real-time analytics.

- **Facial Emotion Recognition:** TensorFlow, Keras, PyTorch
- **Computer Vision & Facial Landmark Detection:** OpenCV, Dlib
- **Natural Language Processing (NLP):** Transformer-based models (BERT, RoBERTa), SpaCy, NLTK
- **Backend Framework:** Flask (Python), Node.js
- **Frontend Framework:** React.js

- **Database:** MongoDB (for unstructured data like logs and embeddings), PostgreSQL (for structured Agile metric data)
- **Cloud & Deployment:** AWS (Amazon Web Services) for scalable computing and storage
- **Containerization & Orchestration:** Docker, Kubernetes

4. PROJECT REQUIREMENTS

4.1 Functional Requirements

1. **Data Integration & Collection:** Connect to and import data from external tools (Jira, Trello, Azure DevOps) via their APIs to collect sprint backlogs, user stories, tasks, defects, and team metrics.
2. **Data Preprocessing & Structuring:** Clean, normalize, and transform raw data into a consistent, structured format for analysis, handling missing values and standardizing units like story points.
3. **Automated Change Detection:** Automatically compare sprint data versions to identify added, removed, or modified items, including changes to story points, priority, dependencies, or assignees.
4. **Change History Maintenance:** Keep a historical log of all requirement changes to support audit trails and impact analysis over time.
5. **Feature Engineering:** Automatically derive quantitative metrics (e.g., backlog churn rate, workload per developer, defect injection rate) from the raw data to be used as model inputs.
6. **Impact Prediction Modeling:** Use machine learning models to predict the impact of changes on four key factors: sprint velocity, schedule/delays, product quality, and team productivity.
7. **Dynamic Priority Scoring:** Calculate and continuously update a composite priority score for each backlog item based on the predicted impacts, with configurable weights for each factor.
8. **Replanning Suggestions:** Generate revised sprint backlog recommendations, suggesting task redistribution, item postponement, or resequencing based on dependencies and priorities.
9. **Dashboard Visualization:** Display data, predictions, and recommendations through an interactive graphical dashboard with charts, metrics, and drill-down capabilities.
10. **Reporting & Export:** Generate and export clear, detailed reports (PDF/CSV) for sprint reviews and stakeholder communication.

4.2 Non-Functional Requirements

1. **Performance:** Process data and generate predictions quickly (e.g., data sync <5 min, predictions <30 sec) to support real-time decision-making.
2. **Reliability:** Operate consistently with high uptime (e.g., 99.5%), gracefully handling errors without crashing.
3. **Security:** Implement authentication, role-based access control (RBAC), and encryption for user credentials and sensitive project data.
4. **Usability:** Feature an intuitive, user-friendly interface that requires minimal training, with clear explanations for all recommendations.
5. **Accuracy:** Ensure prediction models and recommendations are accurate and trustworthy (e.g., >85% precision/recall for key metrics).
6. **Integrability:** Be designed with modular, well-documented APIs to easily connect with and add new third-party tools (Jira, Trello, etc.).
7. **Maintainability:** Be built with code that is easy to update, modify, and extend for new features or model improvements.

4.3 System Requirements

1. **Software Platform:** A web-based application running on a modern server (Linux/Windows) using a runtime like Node.js or Python, accessible via modern web browsers.
2. **Database:** A dedicated database (e.g., PostgreSQL, MySQL, or MongoDB) with sufficient storage and performance for analytical queries on historical project data.
3. **External Dependencies:** Active internet connection and API access to integrated third-party tools (Jira, Trello, Azure DevOps).
4. **Processing Power:** Adequate CPU and RAM resources on the server to handle data processing and machine learning model computations efficiently.

4.4 User Requirements

1. **Product Owner:** To understand the potential impact of a new requirement or priority change before committing it to a sprint, enabling informed trade-off decisions.
2. **Scrum Master / Team Lead:** To receive automatic alerts on disruptive changes and get data-driven replanning options to protect the team's focus and keep the sprint on track.
3. **Development Team:** To gain visibility into how changes affect their workload and potential quality risks, allowing for proactive adjustments and sustainable productivity.
4. **Stakeholders:** To receive clear, quantifiable reports on how change requests impact delivery timelines and quality, setting realistic business expectations.

5. DESCRIPTION OF PERSONAL AND FACILITIES

A detailed project plan has been formulated to ensure the systematic and timely execution of this research. The plan is structured into five sequential phases, encompassing all activities from initial planning to final thesis submission. The following Work Breakdown Structure (WBS) delineates the specific tasks, their ownership, and interdependencies. Subsequently, a Gantt chart provides a clear visual timeline for the project's 7-month duration, highlighting the critical path and parallel development efforts, particularly in the model development phase. This structured approach is designed to facilitate effective project management, ensure comprehensive coverage of all objectives, and mitigate the risk of delays

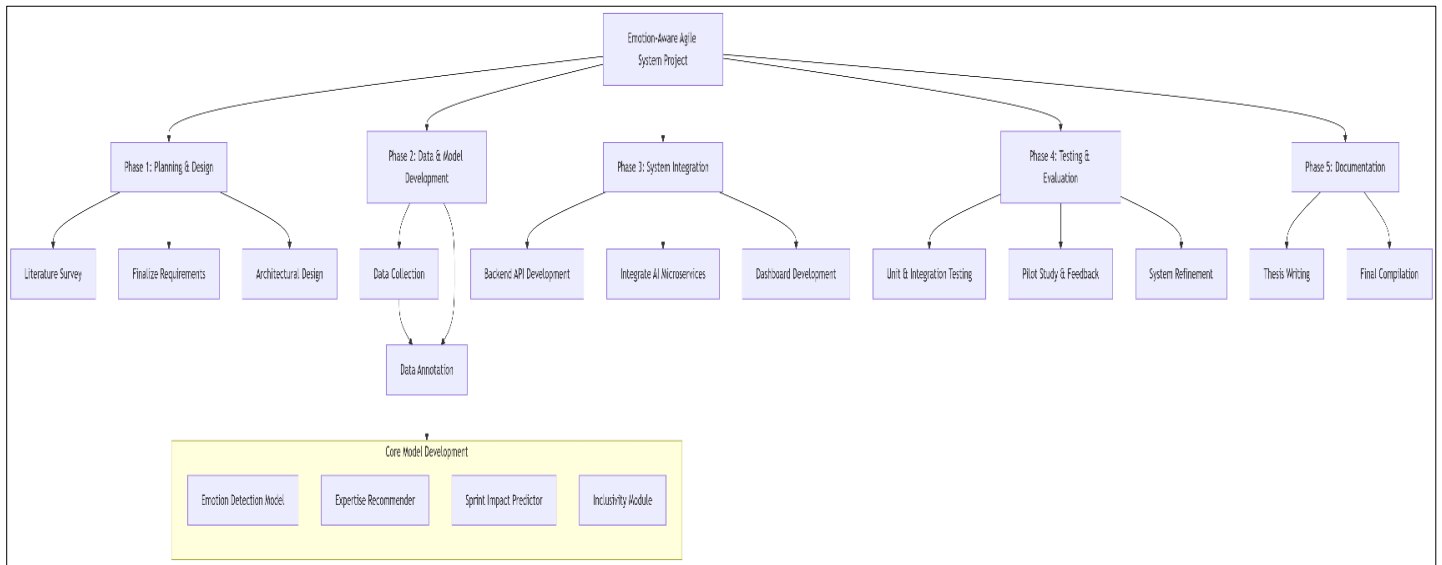


Figure 5.1: Work Breakdown Structure

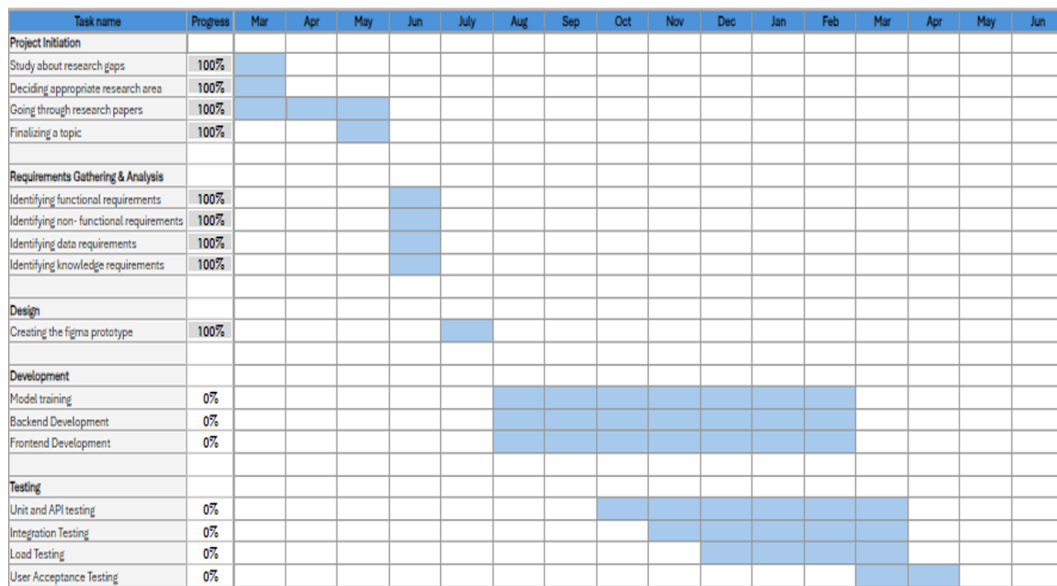


Figure 5.2: Gantt Chart

6. BUDGET AND BUDGET JUSTIFICATION

Category	Description	Estimated Cost (Rs.)
Internet Charges	Monthly broadband charges during project	3,000.00
Development	Expenses for coding tools, IDE licenses, or plugins	5,000.00
Technical Information Learning	Books, tutorials, and online learning materials	4,500.00
Online Resources & Subscriptions	Research platforms, premium accounts for learning	3,000.00
APIs	Paid APIs for integration and testing	6,000.00
Cloud Services	Hosting, compute, and storage services	12,000.00
Databases	Cloud database subscription or license fees	7,500.00

Table 6.1: Overall Budget For System

7. REFERENCES

- [1] M. Choetkiertikul, P. Banyongrakkul, S. Tuarob, C. Ragkhitwetsagul, H. K. Dam, and T. Sunetnant, "Sprint2Vec: A Deep Characterization of Sprints in Iterative Software Development," *IEEE Transactions on Software Engineering*, vol. 51, no. 1, Jan. 2025.
- [2] A. S. Alsharari, W. M. N. W. Zainon, B. A. Mohammed, and M. S. Alsharari, "A Review of Agile Methods for Requirement Change Management in Web Engineering," in *Proc. 2023 Int. Conf. Smart Computing and Application (ICSCA)*, 2023, pp. 1–6, doi: 10.1109/ICSCA57840.2023.10087734.
- [3] M. A. Akbar, J. Sang, Nasrullah, A. A. Khan, S. Mahmood, S. F. Qadri, H. Hu, and H. Xiang, "Success factors influencing requirements change management process in global software development," *Journal of Computer Languages*. [Online]. Available: www.editorialmanager.com/cola/default.aspx
- [4] A. F. B. Arbain, I. Ghani, and W. M. N. W. Kadir, "Agile Non-Functional Requirements (NFR) Traceability Metamodel," *Faculty of Computing, Universiti Teknologi Malaysia, Johor, Malaysia*.
- [5] J. M. Champagne and D. L. Carver, "Measuring Software Change Impact Using Requirement Dependencies," *Division of Computer Science and Engineering, Louisiana State University, Baton Rouge, LA, USA*.
- [6] S. Tariq, A. M. Alghamdi, N. Ahmad, M. U. Ashraf, and A. S. Alfakeeh, "Measuring the Impact of Scope Changes on Project Plan Using EVM," *College of Computer Science and Engineering, University of Jeddah, Saudi Arabia*, 2023.
- [7] B. Sharif, S. A. Khan, and M. W. Bhatti, "Measuring the Impact of Changing Requirements on Software Project Cost: An Empirical Investigation," *College of Electrical & Mechanical Engineering, NUST, Islamabad, Pakistan*, 2023.
- [8] S. Jayatilleke and R. Lai, "A Systematic Review of Requirements Change Management," *Information and Software Technology*, vol. 93, pp. 163–181, Sep. 2017, doi: 10.1016/j.infsof.2017.09.004.
- [9] Graziotin, D., Wang, X., & Abrahamsson, P. (2015). Do feelings matter? On the correlation of affects and the self-assessed productivity in software engineering. *Journal of Software: Evolution and Process*, 27(7), 467–487.
- [10] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, 4171–4186.

- [11] Zhang, Z., Luo, P., Loy, C. C., & Tang, X. (2020). Learning from multiple modalities: Emotion recognition in the wild. *IEEE Transactions on Multimedia*, 22(3), 634–646.
- [12] Li, X., Pfister, T., Huang, X., Zhao, G., & Pietikäinen, M. (2018). A spontaneous micro-expression database: Inducement, collection and baseline. *IEEE Transactions on Affective Computing*, 9(1), 41–54.
- [13] Ekman, P., & Friesen, W. V. (2003). *Unmasking the face: A guide to recognizing emotions from facial clues*. Malor Books.
- [14] Alsaqaf, W., Daneva, M., & Wieringa, R. (2019). Quality requirements in agile software development: A systematic literature review. *International Conference on Agile Software Development*, 29–46.
- [15] Amershi, S., Weld, D., Vorvoreanu, M., Fournery, A., Nushi, B., Collisson, P., ... & Horvitz, E. (2019). Guidelines for human–AI interaction. *Proceedings of CHI 2019*, 1–13.